

GRADE 12 COMPUTER SCIENCE

RELATIONAL DATABASE DESIGN AND IMPLEMENTATION USING MySQL

School Management System

Project Title	School Management System
Student	Srijan Gurung
Class	Grade 12
Database System	MySQL
Database Name	school_management
Project Type	Relational Database Design & Implementation

Declaration: This report presents the database design, normalization, implementation, sample data, and SQL queries required for the Grade 12 Database Management Project.

1. Project Overview

The School Management System is a relational database designed to manage students, teachers, courses, and student enrollments. The system stores structured information and uses primary and foreign keys to maintain relationships and data integrity. The implementation follows MySQL conventions and demonstrates DDL, DML, joins, subqueries, grouping, filtering, and safe data modification.

2. Project Objectives

- Design a normalized relational database up to Third Normal Form (3NF).
- Create tables with appropriate data types, primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and other constraints.
- Insert realistic sample records for testing.
- Demonstrate WHERE, ORDER BY, LIMIT, aggregate functions, GROUP BY, HAVING, subqueries, and joins.
- Demonstrate safe UPDATE and DELETE statements using precise WHERE conditions.

3. ER / Schema Diagram

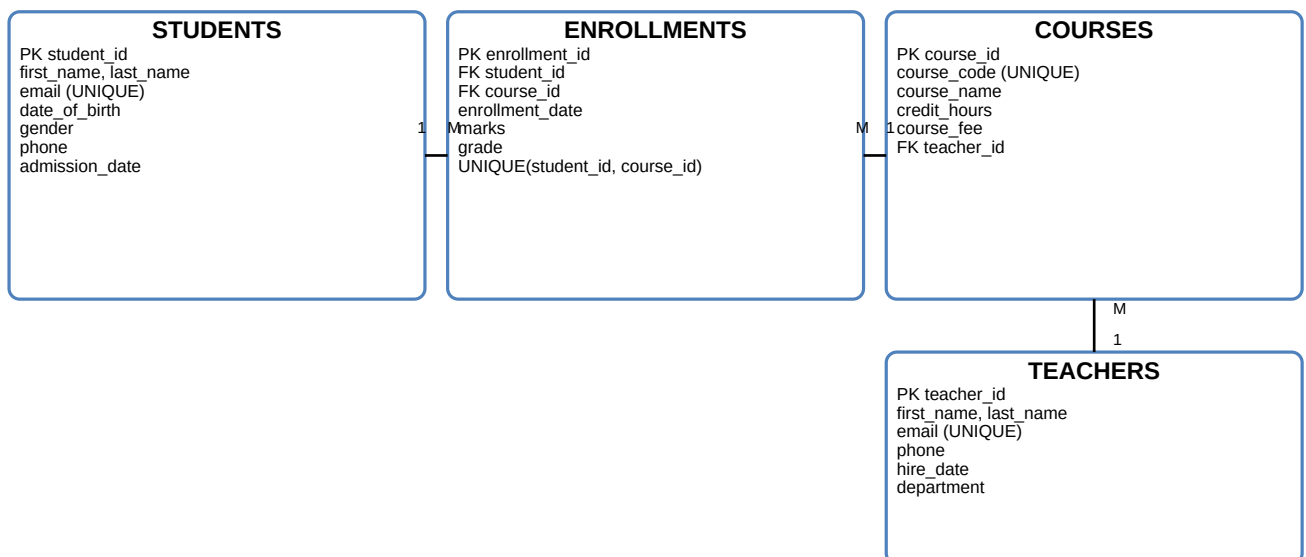


Figure 1: Logical ER/schema design. One student can have many enrollments; one course can have many enrollments; one teacher can teach many courses.

4. Relationship Design

Relationship	Cardinality	Explanation
Students → Enrollments	1 : Many	One student may enroll in multiple courses.
Courses → Enrollments	1 : Many	One course may have multiple enrolled students.
Teachers → Courses	1 : Many	One teacher may teach multiple courses.

5. Table Descriptions

Table	Purpose	Primary Key	Important Foreign Key
teachers	Stores teacher details and departments.	teacher_id	—
students	Stores student personal and admission information.	student_id	—
courses	Stores course details, fees, credits, and assigned teacher.	course_id	teacher_id → teachers
enrollments	Connects students and courses and stores marks/grades.	enrollment_id	student_id → students; course_id → courses

6. Normalization

1NF: Every field contains a single atomic value, and each table has a primary key. Repeating groups are not stored in one column.

2NF: Non-key attributes depend on the whole key. Student details, course details, and teacher details are separated from enrollment records.

3NF: Non-key attributes depend only on their table's primary key. For example, teacher department is stored in teachers rather than repeatedly in courses or enrollments.

7. Constraints and Data Integrity

Constraint	Example	Purpose
PRIMARY KEY	student_id	Uniquely identifies each record.
FOREIGN KEY	enrollments.student_id	Maintains relationships between tables.
UNIQUE	student email, course_code	Prevents duplicate values.
NOT NULL	first_name, email	Ensures required information is present.
CHECK	marks BETWEEN 0 AND 100	Prevents invalid marks.
AUTO_INCREMENT	ID columns	Simplifies record creation and unique IDs.

8. SQL Implementation

The file **database_setup.sql** contains the complete executable MySQL script. It creates the database, creates all four tables, inserts sample data, and executes the required analytical and DML queries.

Database Creation

```
DROP DATABASE IF EXISTS school_management;  
CREATE DATABASE school_management;  
USE school_management;
```

Table Creation Order

The tables are created in parent-to-child order: **teachers** and **students** first, then **courses**, and finally **enrollments**.

Sample Data Summary

Table	Number of sample records
teachers	6
students	10
courses	8
enrollments	10

9. Required Queries and Expected Results

The tables below are expected result previews based on the sample data. For the final submission, run the SQL in MySQL Workbench/terminal and replace these previews with screenshots of the actual MySQL output.

9.1 Basic Retrieval — WHERE, ORDER BY, LIMIT

```
SELECT student_id, first_name, last_name, email  
FROM students  
WHERE gender = 'Male'  
ORDER BY last_name ASC  
LIMIT 5;
```

student_id	first_name	last_name
7	Nabin	Adhikari
1	Aayush	Gurung
9	Roshan	Magar
2	Bibek	Shrestha
4	Sanjay	Tamang

Expected result: first five male students after alphabetical sorting by last name.

9.2 Aggregate Function — COUNT, AVG, GROUP BY, HAVING

```
SELECT c.course_code, c.course_name,  
       COUNT(e.enrollment_id) AS total_students,  
       ROUND(AVG(e.marks), 2) AS average_marks  
FROM courses AS c  
INNER JOIN enrollments AS e ON c.course_id = e.course_id  
GROUP BY c.course_id, c.course_code, c.course_name  
HAVING COUNT(e.enrollment_id) >= 2  
ORDER BY average_marks DESC;
```

course_code	course_name	total_students	average_marks
MATH101	Mathematics	2	87.50
CS101	Computer Science	3	82.00

Expected result: only courses with at least two enrollments are displayed.

9.3 Subquery — Nested Query

```
SELECT student_id, first_name, last_name
FROM students
WHERE student_id IN (
    SELECT student_id
    FROM enrollments
    WHERE course_id IN (
        SELECT course_id
        FROM courses
        WHERE course_fee > 30000
    )
)
ORDER BY last_name, first_name;
```

student_id	first_name	last_name
8	Sneha	Bista
1	Aayush	Gurung
10	Kabita	Poudel
2	Bibek	Shrestha

Expected result: students enrolled in courses whose fee is greater than 30,000.

9.4 INNER JOIN

```
SELECT s.student_id,
       CONCAT(s.first_name, ' ', s.last_name) AS student_name,
       c.course_name, e.marks, e.grade
FROM students AS s
INNER JOIN enrollments AS e ON s.student_id = e.student_id
INNER JOIN courses AS c ON e.course_id = c.course_id
ORDER BY s.student_id;
```

ID	Student	Course	Marks	Grade
1	Aayush Gurung	Computer Science	88.00	A
2	Bibek Shrestha	Computer Science	76.00	B+
3	Pragya Karki	Mathematics	91.00	A+
4	Sanjay Tamang	Mathematics	84.00	A
5	Anusha Rai	Physics	79.00	B+

Expected result: matching student, enrollment, and course information is combined.

9.5 LEFT JOIN

```
SELECT t.teacher_id,
       CONCAT(t.first_name, ' ', t.last_name) AS teacher_name,
       c.course_name
FROM teachers AS t
LEFT JOIN courses AS c ON t.teacher_id = c.teacher_id
ORDER BY t.teacher_id, c.course_name;
```

Teacher ID	Teacher	Course
1	Anita Sharma	Computer Science
1	Anita Sharma	Web Technology
2	Bikash Thapa	Mathematics
3	Nisha Karki	General Science
3	Nisha Karki	Physics
4	Ramesh Adhikari	English
5	Sita Gurung	Social Studies
6	Prakash Rai	Accounting

Expected result: every teacher is retained even if a teacher has no matching course.

9.6 UPDATE Statement

```
UPDATE students
SET phone = '9812999999'
WHERE student_id = 10;
```

Purpose: safely updates one student's phone number using the primary key in the WHERE clause.

9.7 DELETE Statement

```
DELETE FROM enrollments
WHERE enrollment_id = 10;
```

Purpose: safely deletes one enrollment record using its unique primary key.

10. Execution Notes

- Open MySQL Workbench or the MySQL terminal.
- Open and execute database_setup.sql from top to bottom.
- Refresh the school_management schema and verify all four tables.
- Run each Phase 3 query individually and capture screenshots of the actual MySQL results.
- Keep database_setup.sql unchanged as the evaluator's executable script.

11. Conclusion

The School Management System demonstrates a complete relational database workflow from design and normalization to implementation, sample data insertion, querying, joins, subqueries, aggregation, and safe data modification. The schema separates student, teacher, course, and enrollment information to reduce redundancy and maintain data integrity through primary and foreign keys.

Submission files: project_report.pdf and database_setup.sql

Prepared for Grade 12 Computer Science — MySQL Database Management Project